Communication Lower Bounds for Multi-Party Graph Traversal

Quarter 2 Project Report

Ryan Batubara rbatubara@ucsd.edu
Ivy Hawks mahawks@ucsd.edu
Darren Ho dah103@ucsd.edu
Ciro Zhang ciz001@ucsd.edu

Mentor: Dr. Shachar Lovett
slovett@ucsd.edu

Abstract

A large number of computational problems can be modelled as communication games: For example, given a graph, two players each have a subgraph they know is traversable, and wish to collaboratively construct a path from one vertex to another. Communication complexity studies such situations by creating asymptotic bounds on the minimum communication necessary to complete these tasks. In this report, we prove communication lower and upper bounds for a variety of graph traversal problems, such as the example above. We also review potential applications of these bounds, such as for modelling real-world scenarios and designing algorithms.

Website: https://rybplayer.github.io/capstone/
Repository: https://github.com/rybplayer/DSCCapstone

1 Introduction

1.1 A motivating problem

Graphs are fundamental data structures for modelling relationships between entities. For example, an airline may choose to store their flights as a graph, whose vertices are destinations and directed edges are flights between destinations. Many well-known algorithms make this a feasible data structure for the airline. Now suppose a user wishes to plan flights, perhaps with transits, from point A to B. This user may need to consider a large variety of airlines, each with their own graph. How can the user do this?

Clearly, the user can get the full database of all flights, and compute the desired sequence of flights on their own. But this is obviously costly in many regards. We want to minimize the amount of processing and communication between these parties while still giving the user the desired result. Indeed, there exists a variety of algorithms for this in the literature.

In this report, we ask an adjacent question: What is the minimum amount of communication necessary for the user to complete this task, without considering the local storage and computational cost to each party? Yao's communication complexity model formalizes this idea, allowing us to prove algorithmic lower bounds for situations like the above. These algorithmic lower bounds are concrete, mathematical claims that show we cannot asymptotically reduce communication beyond a certain point.

1.2 Why communication is the bottleneck

In the model we consider, each party has unlimited local computation and we only wish to minimize the communication between them. In the airline routing example above, this restriction is quite realistic: airline servers and user computers can easily compute routes given the data. However, neither can afford to broadcast or receive all states due to bandwidth limitations, especially if there are multiple requests. The communication cost, not the local computation, is the bottleneck.

Communication complexity makes this separation explicit. In particular, it shows that even when each party can compute arbitrarily powerful local summaries, some global questions still require a large number of bits to be exchanged. For example, a route may require flights from multiple airlines. Thus communication is not just a practical nuisance, but an inherent barrier that be designed around.

1.3 Why communication bounds matter

These lower bounds have a variety of applications for algorithm, system and software design. An algorithm's complexity could be described as an upper bound on the amount of communication between parties. If communication complexity provides a corresponding lower bound, this means this algorithm is tight and its communication cannot asymptoti-

cally be improved. For system designers, there is often a lot of choice in how communication is routed through a network. For example, a central server could manage all communications, or a distributed network may be used. Communication complexity can provide lower bounds for each of these scenarios, for instance by proving that distributed networks are asymptotically no better than a centralized communication server. Communication complexity can also inform software decisions, by providing insight into how to manage the bits of communication between parties.

1.4 Communication complexity basics

We briefly define the various measures of communication complexity used in this report. Interested readers should see Rao and Yehudayoff (2020) or Roughgarden (2015) for a more thorough introduction. Let $f : X \times Y \to \{0,1\}$ be a function. In the two-party model, Alice receives $x \in X$ and Bob receives $y \in Y$, and they wish to compute $f(x,y)$. A *deterministic protocol* is a binary decision tree whose nodes are labeled by which party speaks and whose edges are labeled by bits 0 or 1. The *transcript* $\pi(x,y)$ is the sequence of bits exchanged for a particular input x and y. The protocol must output $f(x,y)$ at a leaf. The *deterministic* communication complexity $D(f)$ is the minimum, over all correct protocols, of the maximum transcript length (or equivalently, protocol tree depth) over all inputs.

A *nondeterministic protocol* for f has two parts: First, an external party has access to x and y, and provides advice a to both players. Then, the players, with access to a and their respective input, communicate to decide whether to output 0 or 1. Let $\Pi(x,y,a)$ denote the output of the protocol, which we say is correct if

- If $f(x,y) = 1$, then $\exists a$ with $\Pi(x,y,a) = 1$.
- If $f(x,y) = 0$, then $\forall a$, $\Pi(x,y,a) = 0$.

The cost of Π is the sum of a and any communication between players. The *nondeterministic* cost of f, denoted $N^1(f)$, is the minimal cost of a protocol computing f. The *co-nondeterministic* cost of f is the nondeterministic cost for $1-f$.

Randomized protocols allow parties to use random coins to reduce communication. A public-coin protocol assumes a shared random string, while a private-coin protocol lets each party sample independently. Fixing the randomness yields a deterministic protocol, so a randomized protocol is a distribution over deterministic protocols. The *randomized* communication complexity $R_\varepsilon(f)$ is the minimum communicated bits needed to compute f with error at most ε on every input. Public-coin ($R^{pub}_\varepsilon(f)$) and private-coin ($R_\varepsilon(f)$) models are equivalent up to small overhead, and randomness often trades a small probability of error for large savings in communication. Let $R(f) = R_c(f)$ with $c = 1/3 < 1/2$.

1.5 Communication complexity basic examples

Two classic communication complexity problems are equality and disjointness. Equality asks whether $x = y$ for $x, y \in \{0,1\}^n$, and disjointness asks whether $X \cap Y = \emptyset$ for subsets

of $[n]$. Both are useful for a range of lower-bound techniques, so we go through their communication complexity bounds below:

Claim 1. *Equality, denoted* $\mathsf{EQ} : X \times Y \to \{0,1\}$*, outputs* 1 *if* $x \in X$ *and* $y \in Y$ *have* $x = y$*, and* 0 *otherwise.* EQ *has* $D(\mathsf{EQ}) = n$*,* $N^1(\mathsf{EQ}) = n$*,* $N^0(\mathsf{EQ}) = \Theta(\log n)$*, and* $R_\varepsilon^{pub}(\mathsf{EQ}) = O(\log(1/\varepsilon))$*.*

Proof. First $D(\mathsf{EQ}) = n$.

- Upper bound: Alice sends all n bits of x to Bob, who compared it to y and outputs.
- Lower bound: Any deterministic protocol partitions the $\{0,1\}^n \times \{0,1\}^n$ matrix M_{EQ} into combinatorial z-rectangles, i.e. regions $R = A \times B \subseteq X \times Y$ where $f(a,b) = z$ for all $a \in A$, $b \in B$, and $z \in \{0,1\}$. Clearly $M_{\mathsf{EQ}} = I_{\{0,1\}^n \times \{0,1\}^n}$. Hence there are 2^n 1-rectangles or at least 2^n leaves in the protocol tree. Thus $D(f) \geq \log_2(2^n) = n$.

Now $N^1(\mathsf{EQ}) = n$. This means we want to certify $x = y$.

- Upper bound: The prover sends x to both players. Both players check if it matches their input. If so, accept.
- Lower bound: Similar to $D(\mathsf{EQ})$, we need 2^n distinct certificates to cover all possible inputs. Hence certificate length $\geq \log_2(2^n) = n$.

Now $N^0(\mathsf{EQ}) = \log(n)$. This means we want to certify $x \neq y$.

- The prover computes an index $i \in [n]$ where $x_i \neq y_i$. This takes $\log(n)$ bits to communicate to Alice and Bob, who can exchange bits at that coordinate and witness if they differ.
- There are n possible indices i. If certificates had length $< \log(n)$, there would be $< n$ certificates which cannot cover all the possibilities.

Now $R_\varepsilon^{pub}(\mathsf{EQ}) = O(\log(1/\varepsilon))$.

- Upper bound: Using shared randomness, pick k strings $r^{(1)}, \ldots, r^{(k)} \in \{0,1\}^n$. For each $j \in [k]$, Alice sends $b_j = \langle x, r^{(j)} \rangle \bmod 2$. Bob computes $\langle y, r^{(j)} \rangle \bmod 2$ and checks equality for all $j \in [k]$. If $x = y$, this always succeeds. If $x \neq y$, each check fails independently with probability $1/2$, since the $r^{(j)}$ are uniformly random. Take $k = \lceil \log_2(1/\epsilon) \rceil$.

We do not prove it, but $R_\varepsilon^{pub}(\mathsf{EQ}) = O(\log(n))$ with only private randomness. □

Claim 2. *Set disjointness, denoted* DISJ*, interprets* $x, y \in \{0,1\}^n$ *as subsets* $X, Y \subseteq [n]$ *with*

$$\mathsf{DISJ}(x,y) = 1 \iff \forall i \in [n], x_i y_i = 0. \tag{1}$$

Then $D(\mathsf{DISJ}) = \Theta(n)$*,* $N^1(\mathsf{DISJ}) = \Theta(n)$*,* $N^0(\mathsf{DISJ}) = \Theta(\log n)$*,* $R(\mathsf{DISJ}) = \Theta(n)$*. Furthermore, if* $|X| = |Y| = k \ll n$*, then* $D(\mathsf{DISJ}) = \Theta(k)$*.*

These bounds are well known in the literature, so we choose to omit the proof for DISJ bounds. Both EQ and DISJ are very useful for *reductions*, where we embed a problem with known bounds in another, to show this other problem cannot be solved too efficiently. In particular, EQ is "maximally complicated" for deterministic protocols and DISJ is "maximally complicated" for randomized protocols, in the sense that $D(\mathsf{EQ}) = n$ and $R(\mathsf{DISJ}) = \Theta(n)$.

2 The Traversal Problem

2.1 Motivating our problem

In this and the following sections, we will define and show communication bounds for a series of problems pertaining to multi-party graph traversal. Please see Appendix A for a variety of case studies that motivates the following problem definitions.

2.2 Problem Statement

We begin with the simplest non-trivial formulation of our problem, inspired by 1.1.

Consider a graph $G = (V, E)$ and subgraphs $G_1, \ldots, G_k \subset G$. Let $G_i = (V_i, E_i)$ for these subgraphs. For now let us suppose G is unweighted and undirected. The subgraphs need not cover G and can overlap. Each player i knows G and G_i, but not the other player's graphs. The goal is, given some starting vertex $v \in V$, and some end vertex $w \in V$ to:

1. Identify if it is possible to construct a path $v, v_1, \ldots, v_l, w$ such that every edge on the path is in $\bigcup_{i=0}^{k} E_i$.
2. If such a path exists, find the minimum number of edges needed to get from v to w, while using only edges belonging to $\bigcup_{i=0}^{k} E_i$.

If some $e = (u, v) \in G_i$, we must have $u, v \in V(G_i)$. But the converse is not always true.

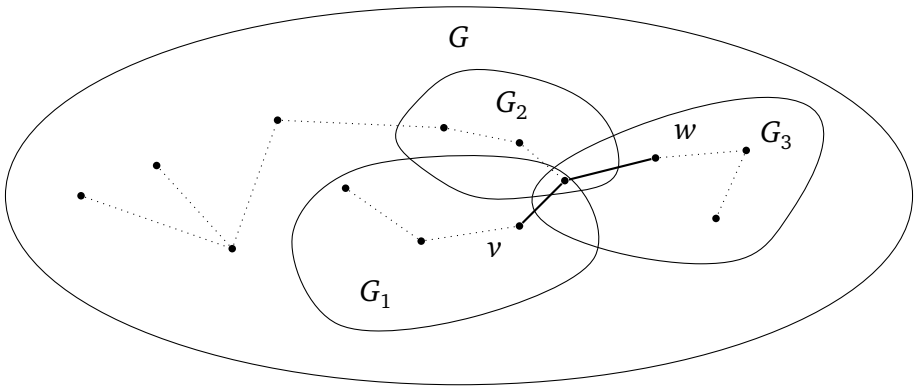

Figure 1: A valid path from v to w in bold. Note that G may contain nodes not belonging to any player. Also, it may be that edge $(u, v) \notin E(G_i)$ though $u, v \in V(G_i)$.

Sometimes, condition 1 fails, as in Figure 2. In such a case, it is sufficient for one party to be convinced that no path exists.

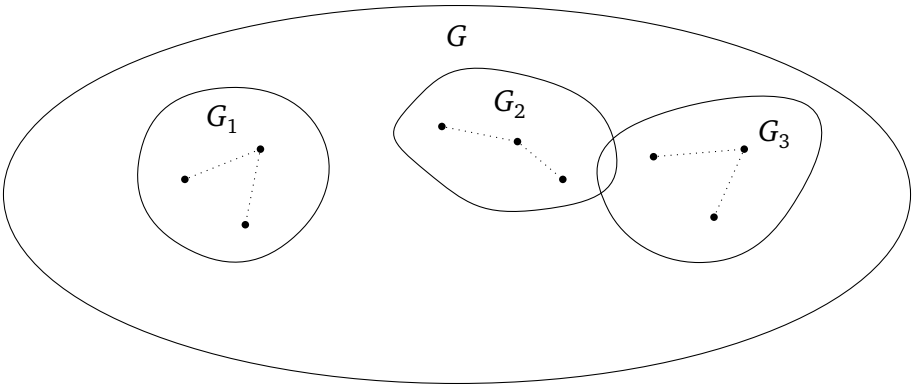

Figure 2: Worst case for case item 1: Disjoint subgraphs G_i

2.3 Lower bound using Equality

Equality problem. In the two-party Equality problem, Alice is given a string $x \in \{0,1\}^n$ and Bob is given a string $y \in \{0,1\}^n$. The goal is to determine whether $x = y$. It is known that Equality has deterministic communication complexity $\Omega(n)$.

Equality gadget. We construct a gadget that enforces equality at a single bit position. The gadget is shown in Figure 3. The gadget consists of two parallel branches.

1. The top branch corresponds to the assignment $x_i = y_i = 1$,
2. The bottom branch corresponds to the assignment $x_i = y_i = 0$.

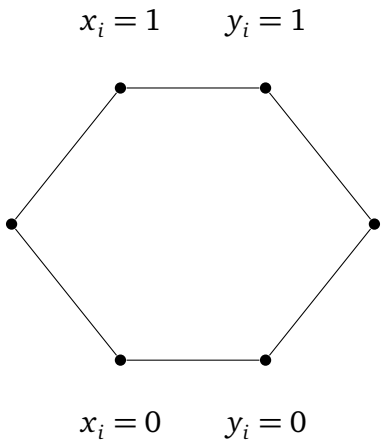

Figure 3: Gadget for proving equality. A path only exists if $x_i = y_i$ for either branch.

Alice enables exactly one entry node of the gadget depending on x_i, and Bob enables exactly one exit node depending on y_i. A path exists through the gadget if and only if Alice and Bob select nodes on the same branch, which holds exactly when $x_i = y_i$.

Reduction to our graph problem. We construct a graph by chaining one copy of this gadget for each index $i \in [n]$ between a global start vertex v and end vertex w. In the resulting graph, there exists a path from v to w if and only if all gadgets are traversable, which occurs if and only if $x_i = y_i$ for all i, i.e., if and only if $x = y$.

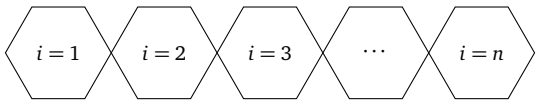

Figure 4: Equality gadgets chained as adjacent hexagonal modules.

Therefore, any protocol that solves the reachability problem on this graph can be used to solve the Equality problem. Since Equality requires $\Omega(n)$ deterministic communication, it follows that our graph problem also requires $\Omega(n)$ deterministic communication.

2.4 Lower bound using Disjointness

Set Disjointness problem. Alice and Bob are both given sets $X, Y \subseteq [n]$ The goal is to determine whether the sets represented by X and Y are disjoint. It is known that Disjointness has deterministic communication complexity $\Omega(n \log n)$. (Note that while this bound seems larger than that of equality, it represents the exact same–Alice sends her entire input to Bob, who then computes the function)

Disjointness gadget. We construct a gadget that enforces disjointness at a single bit position. The gadget is shown in Figure 5. The gadget is constructed so that traversal is possible for all assignments except when both parties contain the element.

1. The top branch corresponds to when x_i has the element but y_i does not
2. The middle branch corresponds to when x_i and y_i do not have the element
3. The top branch corresponds to when y_i has the element but x_i does not

Reduction to our graph problem. We construct a graph by chaining one copy of this gadget for each index $i \in [n]$ between a global start vertex v and end vertex w. In the resulting graph, there exists a path from v to w if and only if all gadgets are traversable, which occurs if and only if x and y does not contain the same element

Therefore, any protocol that solves the reachability problem on this graph can be used to solve the Disjoint problem. Since Disjoint requires $\Omega(n)$ deterministic communication, it follows that our graph problem also requires $\Omega(n)$ deterministic communication.

Non-Induced proofs

Useful as the above proofs are, they require inducing edges along a specific path between the vertices u and v. This has the unfortunate drawback that it does not generalize well

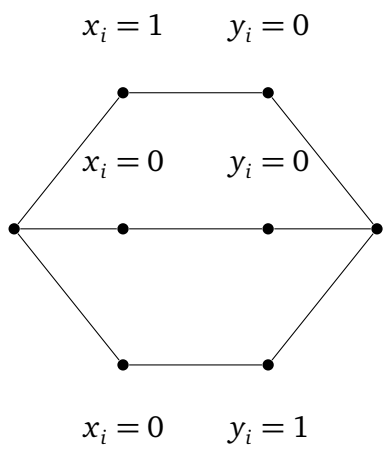

Figure 5: Gadget for proving equality, a path only exists if x_i and y_i do not have the same element

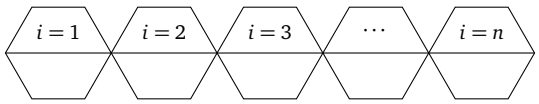

Figure 6: Equality gadgets chained as adjacent hexagonal modules.

to all possible graphs that we might be interested in analyzing. We therefore detail two additional proofs which we can later use in the multiparty setting, without enforcing a specific graph structure.

Connected graph

Given some graph $G(V, E)$, Alice and Bob both receive some subset of vertices. We will impose the constraint that the set of vertices that Alice and Bob each have will form connected subgraphs. We also define a valid path to be one which only uses edges in which both vertices are owned entirely by at least one player, that is (u, v) is a valid edge to traverse iff $u, v \in X$ or $u, v \in Y$. Since path length is not a constraint, Alice and Bob can each generate complete graphs with each of their own internal vertices, and there then exists a valid path between two nodes in one of two cases:

Case 1 Both nodes are in one players input, in which case the problem is trivial.
Case 2 There exists a node in both of the player's input

In the second case, the problem is identifying if $X \cap Y = \emptyset$, and is computed by the Set disjointness protocol, thus creating a bound of $\Omega(n)$ for this problem

2.4.1 Sparse Graphs

Here, we will again assume that Alice and Bob receive some subset of vertices, $X, Y \subseteq [n]$. In this case, we will not attempt to define the edges, and simply assume that $|E| \ll |V|$.

We can define the set of all inputs in which Alice and Bob share exactly one vertex with each other, and otherwise have complete graphs. Alice and Bob will then need to find the one element they have in common in order to determine if there is a valid path. In a set of n vertices, there are then n possible intersecting points, requiring at least $\Omega(n \log n)$ bits to identify one.

3 Other Problems

1. Unbounded Traversal with Common Input ("0-communication" triviality). Model. Fix any class of finite graphs $\mathcal{G}$ and any set of queries $\mathcal{Q}$. An instance is a pair (G, q) with $G \in \mathcal{G}$ and $q \in \mathcal{Q}$. Let $\mathsf{Ans} : \mathcal{G} \times \mathcal{Q} \to \{0, 1\}$ be any decision predicate.

Consider a two-party communication problem in which Alice's input is $x = (G, q)$ and Bob's input is $y = (G, q)$ *identically the same string*. (Equivalently, $y = x$ is promised.)

Define the Boolean function

$$f_{\text{common}}(x, y) \;=\; \mathsf{Ans}(G, q) \quad \text{under the promise } x = y = (G, q).$$

Claim (Triviality). $D(f_{\text{common}}) = 0$ and $N^1(f_{\text{common}}) = N^0(f_{\text{common}}) = 0$.

Proof. Since $x = y$, both parties can compute $\mathsf{Ans}(G, q)$ locally with unbounded computation. Therefore a deterministic protocol that sends no bits exists: each outputs $\mathsf{Ans}(G, q)$. Hence $D(f_{\text{common}}) \leq 0$, so $D(f_{\text{common}}) = 0$.

For nondeterminism, a nondeterministic protocol may also send no bits: each party ignores the certificate and outputs $\mathsf{Ans}(G, q)$. Thus $N^1(f_{\text{common}}) = N^0(f_{\text{common}}) = 0$.

Lower bound. Communication complexity is always ≥ 0 by definition, so the upper bound is tight.

Embedding note. Any nontrivial lower bound requires *private information* (e.g., Alice and Bob see different parts of G, or have different start locations/vision transcripts). Without private inputs, f collapses to a one-input problem, and communication is identically 0.

2. Black Hole Search on a Path (communication $\Theta(\log n)$). Goal. Formalize a decision/identification task where the *only* uncertainty is the black-hole location, and determine the deterministic and nondeterministic communication needed.

Instance. Let P_n be the path graph on vertices $\{1, 2, \ldots, n\}$ with edges $(i, i+1)$. There is an unknown black-hole vertex $b \in [n]$. A probe (agent) that *visits* b is destroyed and sends no further messages. If it never visits b, it survives and may report back.

Communication model. Alice and Bob cooperate and can launch probes from vertex 1. A probe may walk deterministically along the path; upon returning to 1 alive, it can send a *single bit* "alive" to both parties. (If destroyed, no bit is sent.) The *communication cost* is the total number of bits ever successfully sent by surviving probes.

Problem (BH-LOC). The output is the *exact* index $b \in [n]$. Equivalently, define f_b as the function mapping the unknown b to its binary encoding, and ask the parties to output b.

Claim. The deterministic communication complexity to identify b satisfies

$$D(\text{BH-LOC}_n) \;=\; \Theta(\log n).$$

Upper bound (protocol achieving $O(\log n)$ bits). Maintain an interval $I = [L, R]$ known to contain b (initially $[1, n]$). Repeat while $L < R$: let $m = \left\lfloor \frac{L+R}{2} \right\rfloor$. Launch a probe that walks from 1 to m and then returns to 1 along the same path.

- If the probe returns alive, then it never visited b, so $b \notin [1, m]$ and hence $b \in [m+1, R]$. Update $L \leftarrow m+1$.
- If the probe is destroyed, it must have visited b on the way to m, so $b \in [L, m]$. Update $R \leftarrow m$.

Each iteration produces at most one bit of communication (alive/no-alive, where "no-alive" is inferred from silence). The interval size halves each time, so there are at most $\lceil \log_2 n \rceil$ iterations and thus $O(\log n)$ communicated bits. When $L = R$, output $b = L$.

Lower bound (why $\Omega(\log n)$ bits are necessary). Any correct deterministic protocol must distinguish among n possible black-hole locations. Each bit of communication can create at most a factor-2 branching in the transcript. If at most c bits are communicated, there are at most 2^c possible transcripts, hence at most 2^c different outputs can be forced. Correctness for all $b \in [n]$ requires $2^c \geq n$, so $c \geq \lceil \log_2 n \rceil$. Thus $D(\text{BH-LOC}_n) \geq \log_2 n$, matching the protocol up to constants.

Nondeterministic complexity. To certify that the black hole is at location b, a prover can send the index b using $\lceil \log_2 n \rceil$ bits, after which the parties verify by running the above test specialized to b (e.g., test $m = b$ and $m = b-1$). Hence $N^1(\text{BH-LOC}_n) = \Theta(\log n)$. Similarly N^0 is also $\Theta(\log n)$ since the output is an index among n possibilities.

Embedding used for the lower bound. This lower bound is the standard *information* bound: distinguishing n cases requires $\Omega(\log n)$ bits. Equivalently, one can embed the INDEX function by letting b encode an $\log n$-bit string.

3. Graph Isomorphism as Equality under a Rigidity Promise (communication $\Theta(n^2)$).
Inputs. Fix n. Alice receives an n-vertex labeled simple graph G via its adjacency matrix $A \in \{0,1\}^{n \times n}$ (with zeros on the diagonal and symmetry). Bob receives an n-vertex labeled simple graph H via adjacency matrix $B \in \{0,1\}^{n \times n}$. Let $\text{GI}_n(G, H) = 1$ iff G and H are isomorphic (i.e., $A = PBP^\top$ for some permutation matrix P).

Why unrestricted GI is not "just equality". Without restrictions, $\text{GI}_n(G, H) = 1$ may hold even when $A \neq B$ as strings, so it is *not* literally the Equality function on the adjacency matrices.

Promise family (Rigid graphs). Let $\mathscr{R}_n$ be any family of labeled graphs on $[n]$ with *trivial automorphism group*: for every $G \in \mathscr{R}_n$, the only permutation π with $\pi(G) = G$ is the identity. (Equivalently: G has no nontrivial relabeling that preserves adjacency.)

Define the *promised* problem:

$$\mathrm{GI}_n^{\mathscr{R}}(G,H) = \mathrm{GI}_n(G,H) \quad \text{with the promise } G,H \in \mathscr{R}_n.$$

Claim (Triviality-to-equality under the promise). On $\mathscr{R}_n \times \mathscr{R}_n$, isomorphism equals literal equality:

$$\forall G,H \in \mathscr{R}_n, \qquad \mathrm{GI}_n(G,H) = 1 \iff G = H \text{ (as labeled graphs)}.$$

Proof. If $G = H$ as labeled graphs, then clearly they are isomorphic (identity permutation), so $\Rightarrow$ holds. Conversely, suppose $\mathrm{GI}_n(G,H) = 1$. Then there exists a permutation π with $\pi(H) = G$. Apply π^{-1} to both sides: $H = \pi^{-1}(G)$, so G is isomorphic to itself via $\pi \circ \sigma$ for some σ mapping G to H; more simply, since G and H are isomorphic, there exists a bijection φ from $V(G)$ to itself that preserves adjacency between G and G when composing the two isomorphisms $G \to H$ and $H \to G$. This yields a graph automorphism of G. Rigidity of G forces this automorphism to be identity, hence the original isomorphism must be identity on labels, so $G = H$ as labeled graphs.

Deterministic communication complexity. Let $m = \binom{n}{2}$ be the number of independent adjacency bits. Then, under the promise, $\mathrm{GI}_n^{\mathscr{R}}$ is exactly EQ_m (equality on m bits). Therefore

$$D(\mathrm{GI}_n^{\mathscr{R}}) = \Theta(m) = \Theta(n^2).$$

Upper bound protocol (achieves $O(n^2)$). Alice sends her adjacency matrix (or just the upper triangle) to Bob using m bits. Bob checks if his matrix equals Alice's; if yes output 1, else 0. Cost $m = \Theta(n^2)$ bits.

Lower bound (embedding of EQ_m). Equality on m bits has deterministic communication complexity $D(\mathrm{EQ}_m) = m$: Alice must reveal enough to distinguish her m-bit string from all others. Formally, if fewer than m bits are sent, by pigeonhole there exist distinct $x \neq x'$ producing the same transcript, and Bob cannot distinguish (x,x) from (x',x), causing an error on equality. Since $\mathrm{GI}_n^{\mathscr{R}}$ equals EQ_m under the promise, the same lower bound applies.

Nondeterministic bounds. For equality:

$$N^1(\mathrm{EQ}_m) = \Theta(m) \qquad \text{(certificate is the full string)},$$
$$N^0(\mathrm{EQ}_m) = \Theta(\log m) \qquad \text{(certificate is an index of a differing bit)}.$$

Thus

$$N^1(\mathrm{GI}_n^{\mathscr{R}}) = \Theta(n^2), \qquad N^0(\mathrm{GI}_n^{\mathscr{R}}) = \Theta(\log n).$$

Problems embedded to prove bounds. The tight deterministic bound uses an embedding of EQ_m. The nondeterministic 0-certificate bound uses embedding of NEQ_m via an index witness.

4. Multi-agent Sokoban with Shared Instance (communication 0). Sokoban instance. Fix a grid $[W]\times[H]$ with walls $S \subseteq [W]\times[H]$. There are k labeled movable blocks with initial positions $B_1, \ldots, B_k \in ([W] \times [H]) \setminus S$ and k labeled goal cells $G_1, \ldots, G_k \in ([W] \times [H]) \setminus S$. There are k agents with initial positions $A_1, \ldots, A_k$. A move allows an agent to step to a neighboring non-wall cell; if it steps into a block cell, it may push the block one step forward provided the destination cell is empty and non-wall. (Any agent may push any block.)

Winning condition. The players win iff there exists a finite sequence of legal moves after which the set of block positions equals the set of goal positions:

$$\{B_1^{\text{final}}, \ldots, B_k^{\text{final}}\} = \{G_1, \ldots, G_k\}.$$

(Equivalently, blocks can be assigned bijectively to goals at the end; labels do not matter.)

Communication problem. Alice's input and Bob's input are *identical*: the entire Sokoban instance

$$x = y = (W, H, S, (A_i)_{i=1}^k, (B_i)_{i=1}^k, (G_i)_{i=1}^k).$$

Define

$$f_{\text{Soko}}(x, y) = 1 \iff \text{the instance is solvable (a winning move sequence exists).}$$

Claim (Triviality). $D(f_{\text{Soko}}) = 0$ and $N^1(f_{\text{Soko}}) = N^0(f_{\text{Soko}}) = 0$.

Proof. Because $x = y$, both parties have the *entire* instance. With unbounded computation, each can decide solvability locally (e.g., by exhaustive search over the finite state space). Hence there is a protocol with zero communication: each outputs the computed answer. Thus $D(f_{\text{Soko}}) \leq 0$, so $D(f_{\text{Soko}}) = 0$.

For nondeterminism, likewise communication is unnecessary: each party can ignore any certificate and compute the answer directly. So $N^1 = N^0 = 0$.

Tie-breaking discussion (not needed for solvability). If the task were instead to *execute* a solution without communication, the agents can still pre-agree on a deterministic rule to avoid symmetry (e.g., lexicographic priorities over agents/cells), because the full instance is common knowledge; but for the *decision* version, this is irrelevant.

Lower bound. Again, communication is always ≥ 0, so the bound is tight.

Embedding note. To force large communication, one must distribute the instance (e.g., each party sees disjoint subsets of walls/blocks/goals, or private observation transcripts). Under such partitions, one can embed DISJ or EQ by encoding bits into whether certain corridors/locks exist, producing $\Omega(n)$ or $\Omega(n^2)$ lower bounds. Without private inputs, the decision problem has 0 communication complexity.

References

Rao, Anup, and Amir Yehudayoff. 2020. *Communication Complexity: and Applications*. Cambridge University Press

Roughgarden, Tim. 2015. "Communication Complexity (for Algorithm Designers)." [Link]

Appendices

A Multi-Party Graph Traversal Case Studies

Here, we survey a variety of real-world case studies where communication was a major bottleneck.

Large-scale transportation and infrastructure disruptions create a natural model for multi-party graph traversal. Consider the 2010 European volcanic ash shutdown: air-navigation service providers, airports, and airlines each control disjoint portions of airspace and schedules. Rerouting passengers is a shortest-path problem in a layered graph whose feasible vertices and edges change over time. No party can broadcast its full real-time state, both for operational and privacy reasons, so route feasibility must be computed with limited information exchange.

Similar graph-traversal constraints appear in urban transit shutdowns and phased restorations, where distinct agencies own stations and tunnels. After Hurricane Sandy, service availability evolved over weeks, and agencies coordinated alternate routes under partial knowledge of what infrastructure was open. Highway detours after the I-95 bridge collapse in Philadelphia illustrate the same phenomenon in a road network: different operators own highway segments and toll roads, and a single failed cut edge forces traffic onto alternative paths with asymmetric knowledge of capacities.

The maritime and logistics examples are equally instructive. The 2021 Suez Canal blockage forced carriers to reroute around the Cape of Good Hope or wait, a decision that depends on schedule and capacity information held privately by shipping lines and port operators. When the Baltimore Key Bridge collapsed, access to the port was blocked and cargo was diverted to alternate ports, again requiring coordination across a sea–port–land graph with private operational constraints.

Communication bottlenecks become even sharper in critical infrastructure networks. During the 2021 Texas ERCOT blackout, grid operators and utilities effectively solve a constrained flow and connectivity problem as generation and transmission nodes go offline. The Colonial Pipeline cyber disruption similarly forced fuel logistics to reroute through alternate depots and terminals with incomplete visibility. Finally, the 2021 Meta backbone withdrawal and the 2016 Dyn DNS attack show that digital services rely on reachability in routing and dependency graphs, where policy and security constraints limit information sharing across administrative domains.

These cases motivate a theoretical study of multi-party graph traversal: agents must collaboratively decide reachability and shortest paths while their private subgraphs, capacities,

or weights are not fully shareable. The fundamental question is how much communication is necessary, and how randomized protocols, sketches, or partial disclosures can reduce the burden while preserving correctness.